P2P Car Rental System

Final Report

Course	MSE800 Professional Software Engineering
Institution	Yoobee College of Creative Innovation
Assignment	Assignment 1 - Object-Oriented Programming (30%)
Document	Final Report
Version	1.0.0
Date	May 2026
Author	[Your Name]

A peer-to-peer car rental marketplace built in Python, demonstrating object-oriented programming principles, design patterns, and professional software engineering practices.

Abstract

The **P2P Car Rental System** is a command-line peer-to-peer marketplace developed in Python that connects car owners with renters. Unlike traditional car rental companies that own a fleet, the platform owns no vehicles and earns revenue through a 15% commission per booking. Three actors — Owner, Renter, and Admin — interact with the system through role-specific dashboards.

The system demonstrates all four pillars of object-oriented programming (encapsulation, inheritance, abstraction, polymorphism) and applies three design patterns (Singleton, Factory, Strategy). It uses a 4-layer architecture (Presentation → Service → Model → Data Access) backed by SQLite. The design supports multiple environments (dev/test/prod) via configuration, includes database indexes for performance, and is documented with eight UML diagrams plus a separate Maintenance & Support Plan.

Keywords:

Object-Oriented Programming · Design Patterns · Python · SQLite · Peer-to-Peer · UML · Singleton · Factory · Strategy · Software Architecture · CLI · Layered Architecture

Table of Contents

1. Introduction

1.1 Project Background

The car rental industry has been transformed by the peer-to-peer (P2P) model, pioneered by platforms like Turo in the US and similar services worldwide. Instead of a company owning a fleet, individuals list their personal vehicles and earn money when others rent them. This model offers wider vehicle variety, lower prices, and gives owners a way to monetize an asset that otherwise sits idle most of the time.

1.2 Problem Statement

Building a P2P platform requires careful attention to trust, safety, pricing, and matching. The system must:

- **Verify users** before they can transact (KYC) to prevent fraud.
- **Moderate listings** to keep low-quality or fake cars off the platform.
- **Handle bookings** with date conflicts, pricing, insurance options, and approvals.
- **Build trust** through reviews after completed rentals.
- **Process claims** when damages occur, with fair adjudication.

1.3 Project Scope

This project delivers a working Python CLI application that covers the full lifecycle: registration, KYC verification, listing creation, search & filter, booking with optional insurance and discount, owner approval, reviews, and insurance-claim handling. The system is designed for educational purposes — it does not integrate real payment gateways or email infrastructure, but its architecture is production-ready and could be extended in future iterations.

1.4 Objectives

Objective	How it is achieved
Demonstrate OOP principles	Abstract User class with Owner/Renter/Admin subclasses; private attributes; polymorphic dashboard method.
Apply design patterns	Singleton (DatabaseManager), Factory (UserFactory), Strategy (InsuranceStrategy).
Use a layered architecture	4 distinct layers: Presentation, Service, Model, Data Access.
Provide robust error handling	Input validators, try/except around all DB calls, parameterized SQL queries.
Support multiple environments	config.py reads APP_ENV variable to switch between dev / test / prod.
Document with UML	8 UML diagrams: ERD, Use Case, Class, Sequence, and 4 Activity diagrams.

2. System Requirements

2.1 Hardware Requirements

Component	Minimum	Recommended
CPU	1 GHz, any modern	2+ GHz, dual-core
RAM	512 MB	2 GB
Disk	50 MB free	500 MB free
Display	80-column terminal	Modern terminal emulator

2.2 Software Requirements

Software	Version	Purpose
Python	3.8 or higher	Runtime environment
pip	Built-in	Package installer
SQLite	Bundled with Python	Database (no manual install)
tabulate	0.9.0	Pretty CLI tables
colorama	0.4.6	Cross-platform colored output
Operating System	Windows 10+, macOS 10.14+, or Linux	Cross-platform compatible

Note: No external database server is required. SQLite stores all data in a single file (*database/car_rental.db*) created automatically on first run.

3. Installation

Installation takes about two minutes. Follow these three steps:

Step 1: Extract the zip

Extract **p2p_car_rental.zip** to any folder of your choice. The extracted folder is the project root.

Step 2: Create a virtual environment (recommended)

```
# Create the virtual environment
python -m venv venv

# Activate it (Windows)
venv\Scripts\activate

# Activate it (macOS / Linux)
source venv/bin/activate
```

Step 3: Install dependencies

```
# From inside the project folder, with venv active:
pip install -r requirements.txt
```

That installs *tabulate* and *colorama*. The system is now ready to run.

3.1 Running the Application

```
python main.py
```

On first run, the system automatically:

- Creates the SQLite database file (*database/car_rental.db*).
- Sets up all tables, indexes, and constraints.
- Seeds the default admin account.
- Displays the main menu (Login / Register / Exit).

4. Configuration

All configuration lives in **config.py**, the single source of truth for application settings. The active environment is selected via the *APP_ENV* environment variable (default: dev).

4.1 Environments

Environment	Database	Use case
dev (default)	database/car_rental.db (file)	Development; persists between runs
test	:memory: (in-RAM SQLite)	Unit testing; wiped every run
prod	database/car_rental_prod.db	Production deployment

4.2 Switching Environments

```
# macOS / Linux
export APP_ENV=test
python main.py

# Windows (CMD)
set APP_ENV=test
python main.py

# Or inline (Linux/macOS)
APP_ENV=test python main.py
```

4.3 Configurable Constants

Constant	Default	Description
PLATFORM_FEE_PERCENT	15%	Commission per booking
LONG_TERM_DISCOUNTS	7d=10%, 30d=20%	Rental length discount tiers
DISCOUNT_CODES	SAVE10, WELCOME20, SUMMER15	Demo promo codes
DEFAULT_ADMIN_EMAIL	admin@p2p.com	Seed admin login
DEFAULT_ADMIN_PASSWORD	admin123	Seed admin password

5. User Guide

The system supports three roles. Each role sees a different dashboard with role-specific menu options. The recommended first-time workflow is:

#	Step	As role
1	Login as default admin (admin@p2p.com / admin123)	Admin
2	Register a new owner account; logout	Anonymous
3	Register a new renter account; logout	Anonymous
4	Login as admin; verify both new users	Admin
5	Login as owner; add a car listing	Owner
6	Login as admin; approve the car	Admin
7	Login as renter; search cars, make a booking	Renter
8	Login as owner; approve the booking	Owner
9	Login as renter; submit a review after rental	Renter

5.1 As an Owner

- **Add car listing:** Enter make, model, year, mileage, location, and daily rate. Listing starts as *pending* until admin approval.
- **Manage bookings:** View pending requests; approve or reject each. Approval is final and blocks those dates on your car calendar.
- **View earnings:** Cumulative income from approved/completed bookings.

5.2 As a Renter

- **Search cars:** Filter by location and max price per day.
- **View details:** See full car info, daily rate, and minimum / maximum rental periods.
- **Make booking:** Choose dates, optionally select insurance (Basic $5/day, Standard $10/day, Premium $15/day), optionally apply a discount code, then confirm.
- **Cancel booking:** Allowed before the rental period starts.
- **Submit review:** Rate the owner 1-5 stars with an optional comment after the rental is complete.

5.3 As an Admin

- **Verify users:** Confirm new registrations are legitimate (KYC).
- **Approve cars:** Reject low-quality or suspicious listings.
- **View bookings & reports:** Platform-wide statistics and revenue.

- **Handle claims:** Adjudicate insurance disputes (approve, partial, reject).
- **Suspend users:** Revoke verification for misbehaving users.

5.4 Demo Discount Codes

Code	Discount
SAVE10	10% off base price
WELCOME20	20% off base price
SUMMER15	15% off base price

6. Features

The system implements the following functional features grouped by user role:

Category	Features
Authentication	User registration, login, password hashing (SHA-256 + salt), session management.
Owner features	Add / remove cars; set pricing and availability; approve / reject booking requests; view earnings dashboard; view profile.
Renter features	Search & filter cars by location and price; view car details; make booking with date selection; cancel booking; submit reviews.
Admin features	Verify users (KYC); approve / reject car listings; view all bookings; handle insurance claims; view platform reports & statistics; suspend users.
Pricing engine	Daily rate × days, with automatic long-term discounts (7+ days = 10%, 30+ days = 20%), optional discount codes, optional insurance fees, and a 15% platform commission.
Insurance plans	Three tiers (Basic $5/day, Standard $10/day, Premium $15/day) implemented with the Strategy design pattern.
Reviews & ratings	Renters rate owners (1-5 stars) after a booking; ratings appear on profile dashboards and influence future booking decisions.
Reports	Admin can see total users, owners, renters, cars, bookings, and platform revenue in a single dashboard.

7. Project Structure

The codebase follows a strict 4-layer architecture. Each folder represents one architectural layer or one cross-cutting concern.

```
p2p_car_rental/
■
■■■ main.py # Application entry point
■■■ config.py # Environment-based configuration
■■■ requirements.txt # Python dependencies
■■■ README.md # User documentation
■
■■■ database/ # Data Access Layer
■ ■■■ db_manager.py # Singleton DB manager
■ ■■■ car_rental.db # SQLite file (auto-created)
■
■■■ models/ # Domain (OOP entities)
■ ■■■ user.py # Abstract User class
■ ■■■ owner.py # Inherits User
■ ■■■ renter.py # Inherits User
■ ■■■ admin.py # Inherits User
■ ■■■ car.py # Car entity
■ ■■■ booking.py # Booking entity
■ ■■■ review.py # Review entity
■
■■■ factories/ # Factory pattern
■ ■■■ user_factory.py
■
■■■ patterns/ # Design patterns
■ ■■■ insurance_strategy.py # Strategy pattern
■
■■■ services/ # Business logic layer
■ ■■■ auth_service.py
■ ■■■ car_service.py
■ ■■■ booking_service.py
■ ■■■ review_service.py
■ ■■■ admin_service.py
■
■■■ ui/ # Presentation (CLI menus)
■ ■■■ main_menu.py
■ ■■■ owner_menu.py
■ ■■■ renter_menu.py
■ ■■■ admin_menu.py
■
■■■ utils/ # Cross-cutting helpers
■ ■■■ security.py # Password hashing
■ ■■■ validators.py # Input validators
■ ■■■ helpers.py # CLI display helpers
■
■■■ docs/ # Documentation
■■■ UML/ # 8 UML diagrams (PNG)
■■■ screenshots/ # CLI screenshots
```

8. System Background (Screenshots)

These screenshots demonstrate the system in action. They show the actual CLI experience seen by users for each role.

8.1 Main Menu

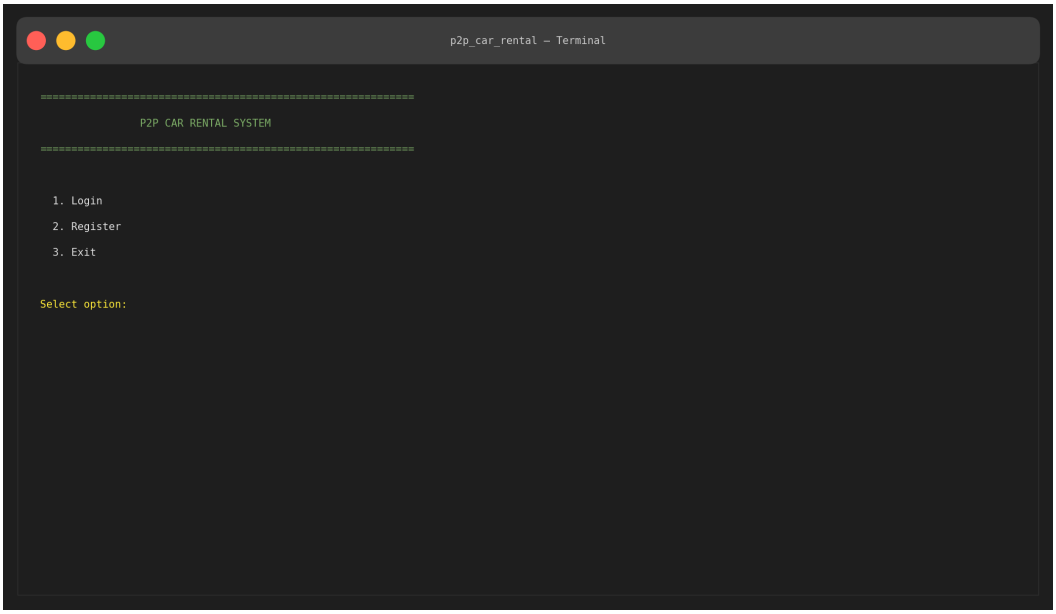

Figure 1: Main menu shown when the application starts.

8.2 Login

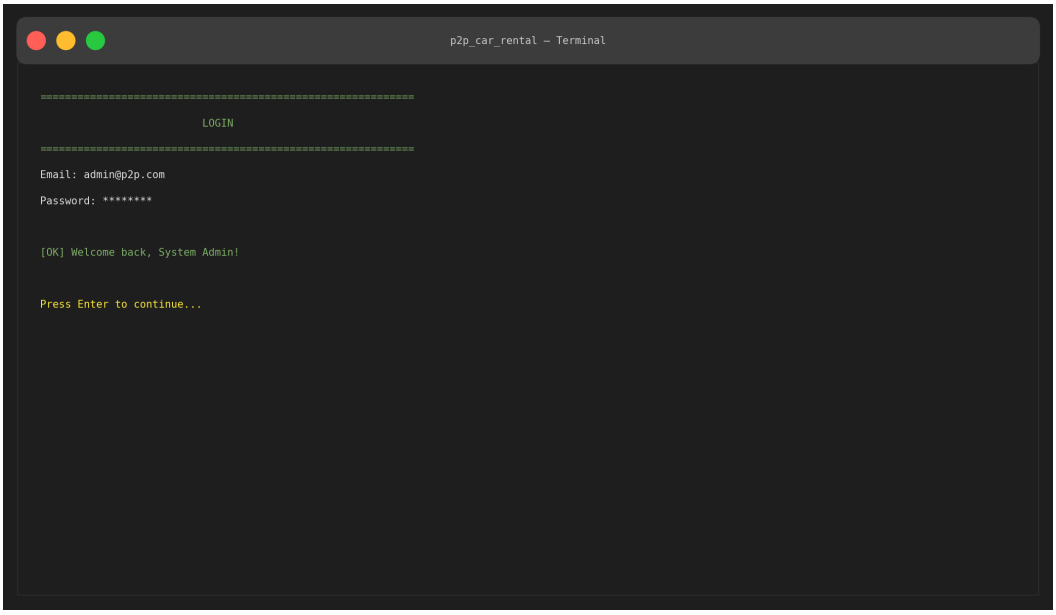

Figure 2: Login flow with successful authentication.

8.3 Owner Dashboard

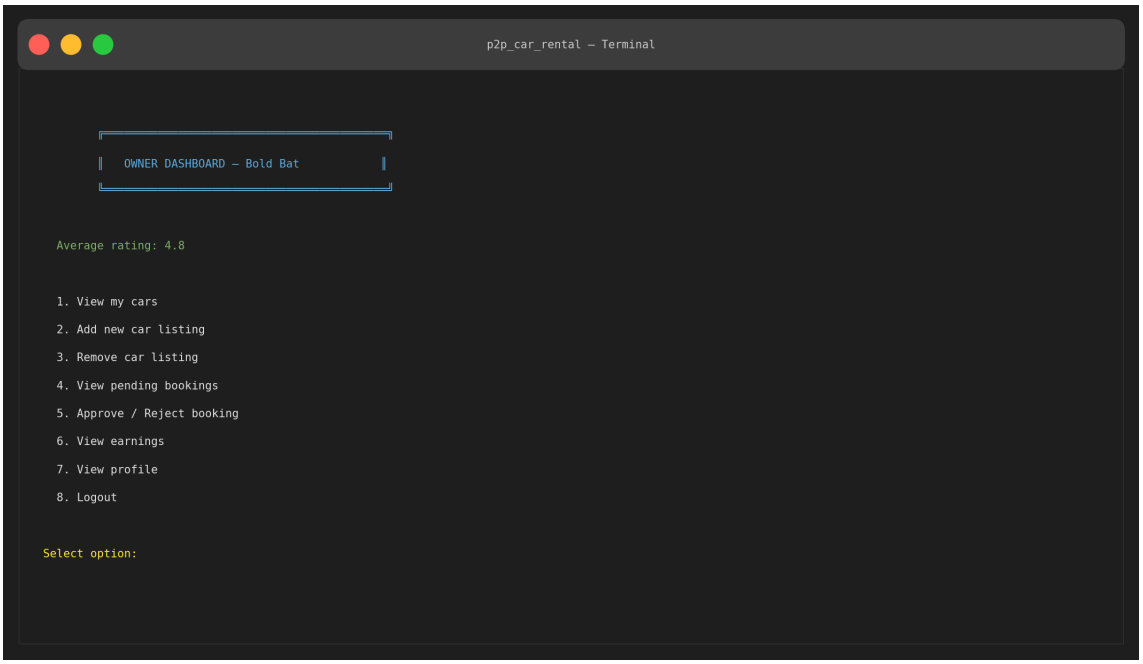

Figure 3: Owner dashboard with role-specific menu options.

8.4 Search Cars

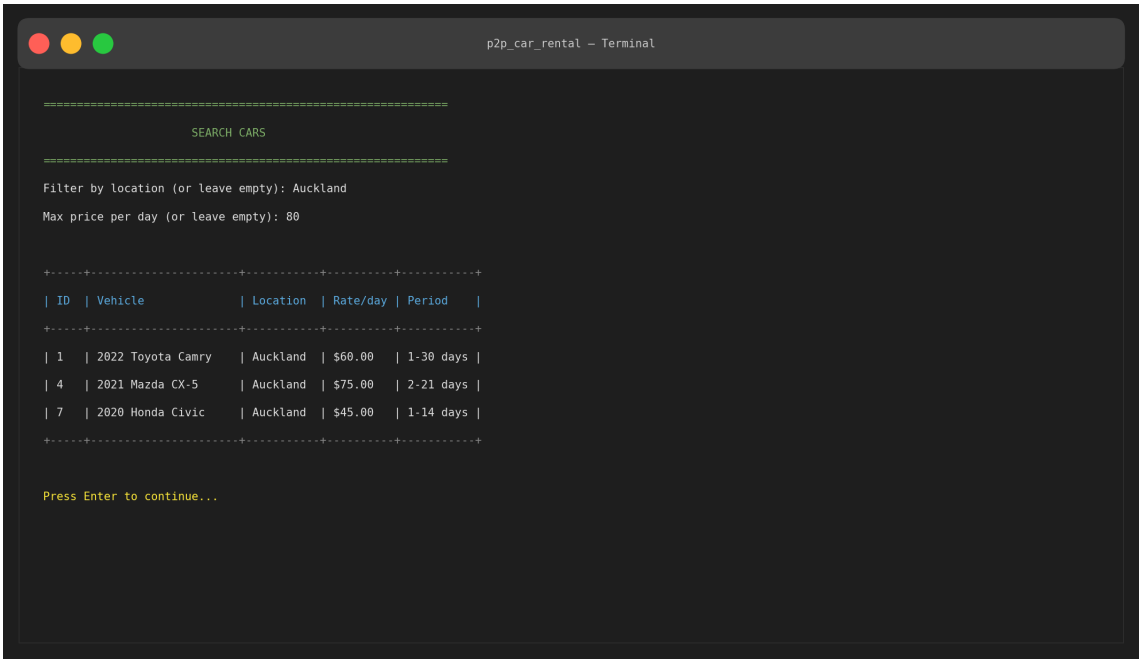

Figure 4: Renter searches available cars with filters.

8.5 Booking Price Breakdown

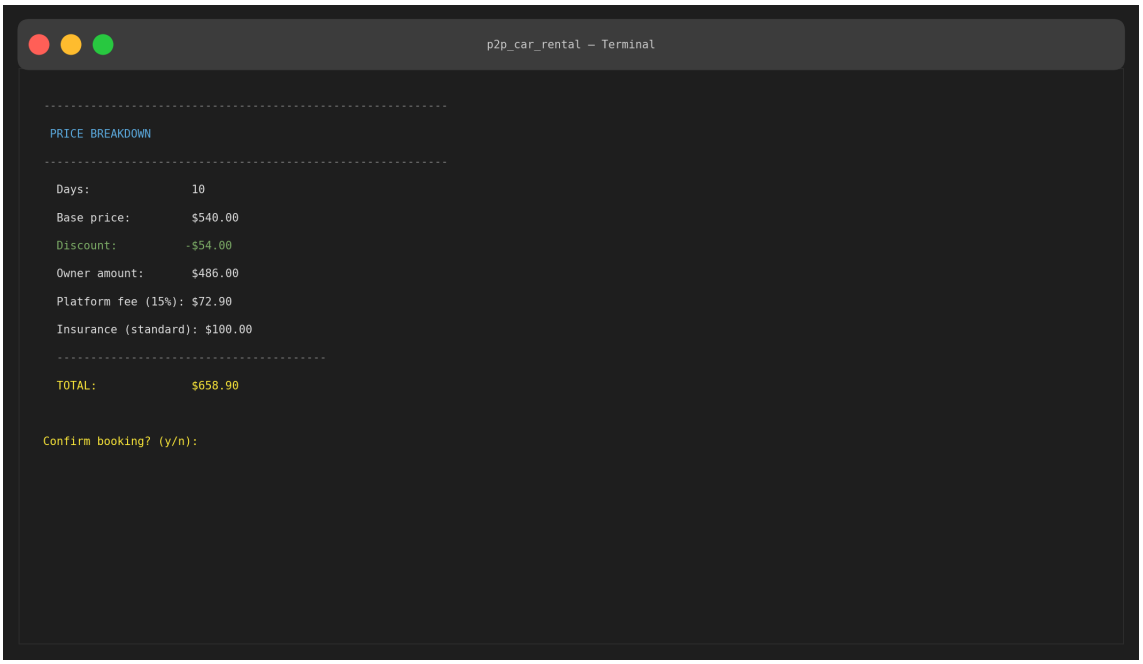

Figure 5: Itemized price breakdown shown before confirming a booking. Demonstrates discount, insurance, and platform fee calculations.

8.6 Admin Platform Reports

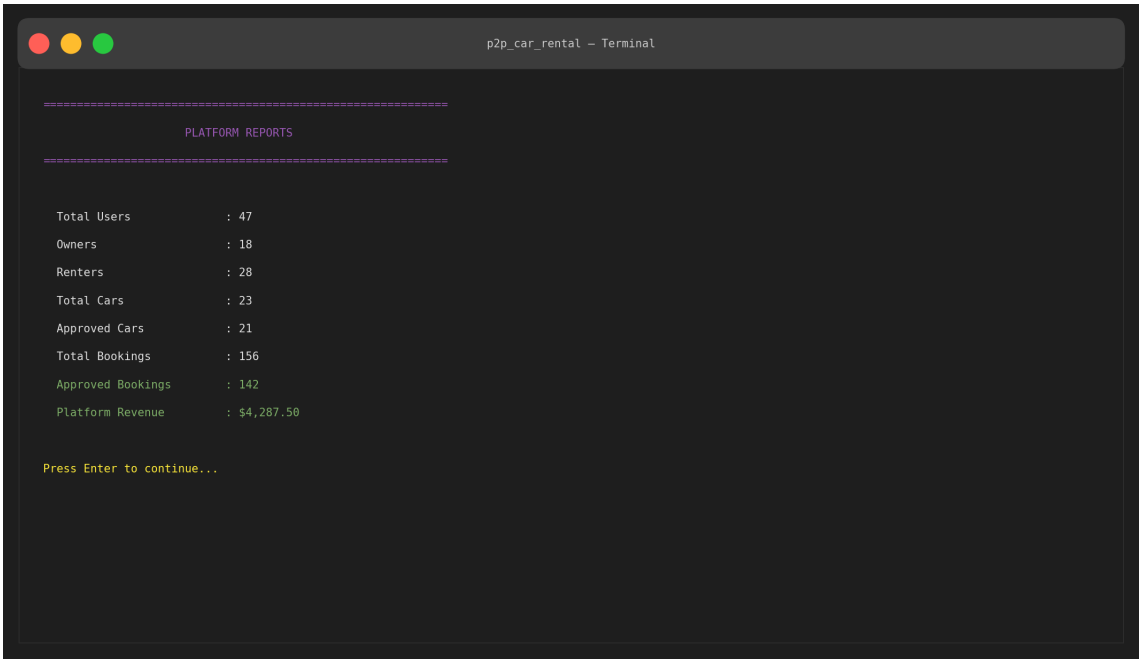

Figure 6: Admin reports showing platform-wide statistics, computed in a single optimized SQL query.

9. Design Patterns

The system applies three classical design patterns from the Gang of Four catalog. Each pattern solves a specific problem and makes the code more maintainable.

9.1 Singleton — DatabaseManager

Problem: Multiple database connections waste resources and risk race conditions on writes.

Solution: Override *__new__* so that all calls to *DatabaseManager()* return the SAME instance.

```
class DatabaseManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialize()
        return cls._instance
```

Benefit: One connection across the whole app. Tables and indexes are created once. Shutdown is centralized.

9.2 Factory — UserFactory

Problem: When loading a user from the database, we need to instantiate Owner, Renter, or Admin depending on the role string. Scattered if/elif chains duplicate this logic.

Solution: Centralize creation in *UserFactory.create(role, ...)*. Adding a new role requires changes in ONE file.

```
class UserFactory:
    @staticmethod
    def create(role, user_id, name, email, ...):
        if role == 'owner':
            return Owner(user_id, name, email, ...)
        elif role == 'renter':
            return Renter(user_id, name, email, ...)
        elif role == 'admin':
            return Admin(user_id, name, email, ...)
```

9.3 Strategy — InsuranceStrategy

Problem: The system offers three insurance plans (Basic, Standard, Premium) with different fees and coverage limits. Hard-coding if/elif inside the booking service would make adding new plans painful.

Solution: Each plan is its own class implementing the InsuranceStrategy interface. The Booking holds a reference and delegates fee calculation.

```
class InsuranceStrategy(ABC):
    @abstractmethod
    def calculate_fee(self, days): pass

class BasicInsurance(InsuranceStrategy):
    daily_fee = 5.0  # $5/day, $2000 coverage

class StandardInsurance(InsuranceStrategy):
    daily_fee = 10.0  # $10/day, $5000 coverage

class PremiumInsurance(InsuranceStrategy):
    daily_fee = 15.0  # $15/day, full coverage
```

Benefit: Adding a new insurance plan is just adding a new class. The booking service code never changes. This follows the *open/closed principle* — open for extension, closed for modification.

9.4 Pattern Summary

Pattern	Used For	Source File
Singleton	Single database connection	database/db_manager.py
Factory	Create User subclasses by role	factories/user_factory.py
Strategy	Pluggable insurance pricing plans	patterns/insurance_strategy.py

10. UML Diagrams

The system is documented with **8 UML diagrams** covering structure, behavior, and process flows. Each diagram is followed by a brief explanation.

10.1 Entity Relationship Diagram (ERD)

The ERD shows the database schema — 5 tables with primary keys, foreign keys, and one unique key constraint.

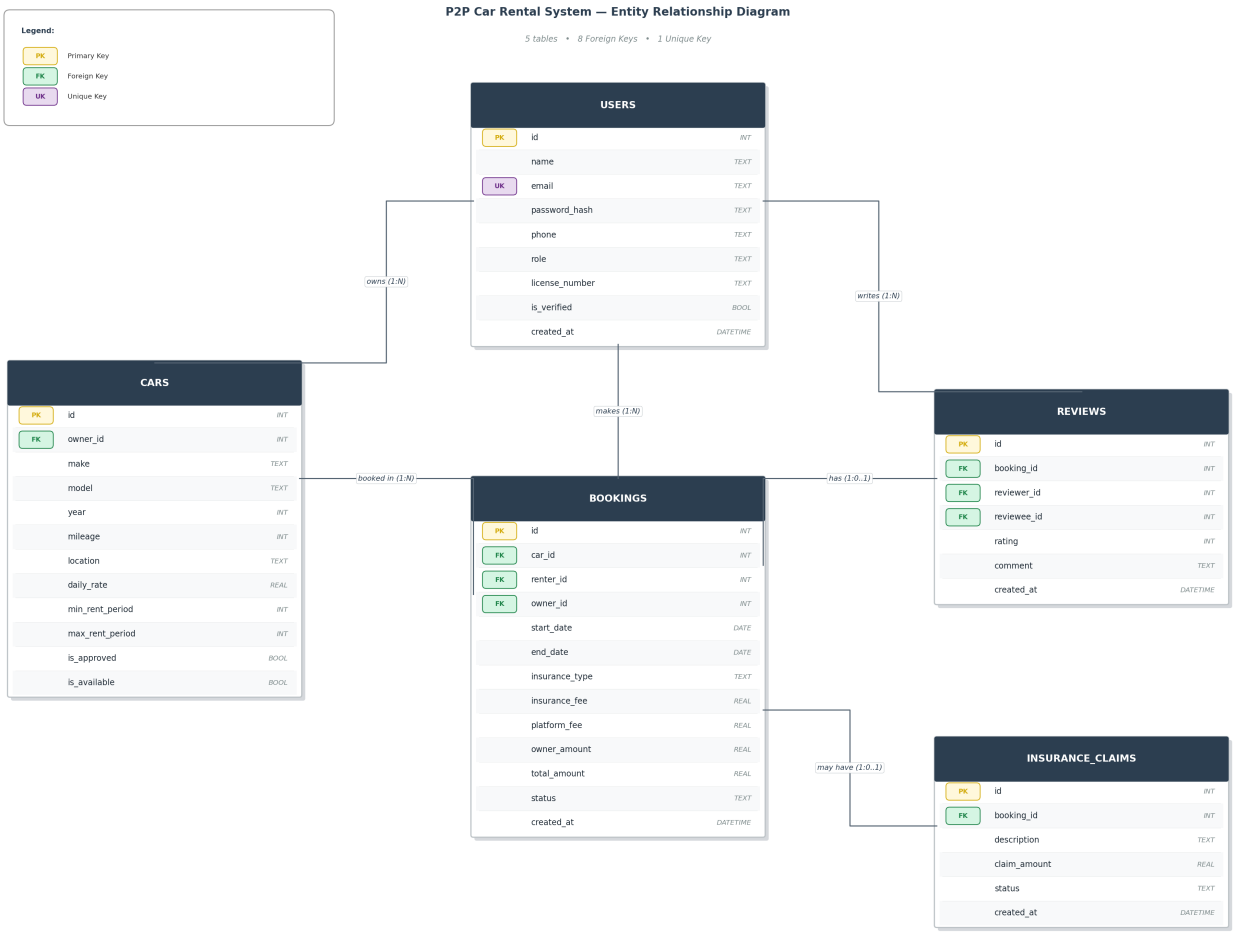

Figure 7: ERD — 5 tables, 8 FKs, 1 UK constraint.

10.2 Use Case Diagram

Captures what each actor can do. Demonstrates «include» (mandatory subtasks) and «extend» (optional add-ons) relationships.

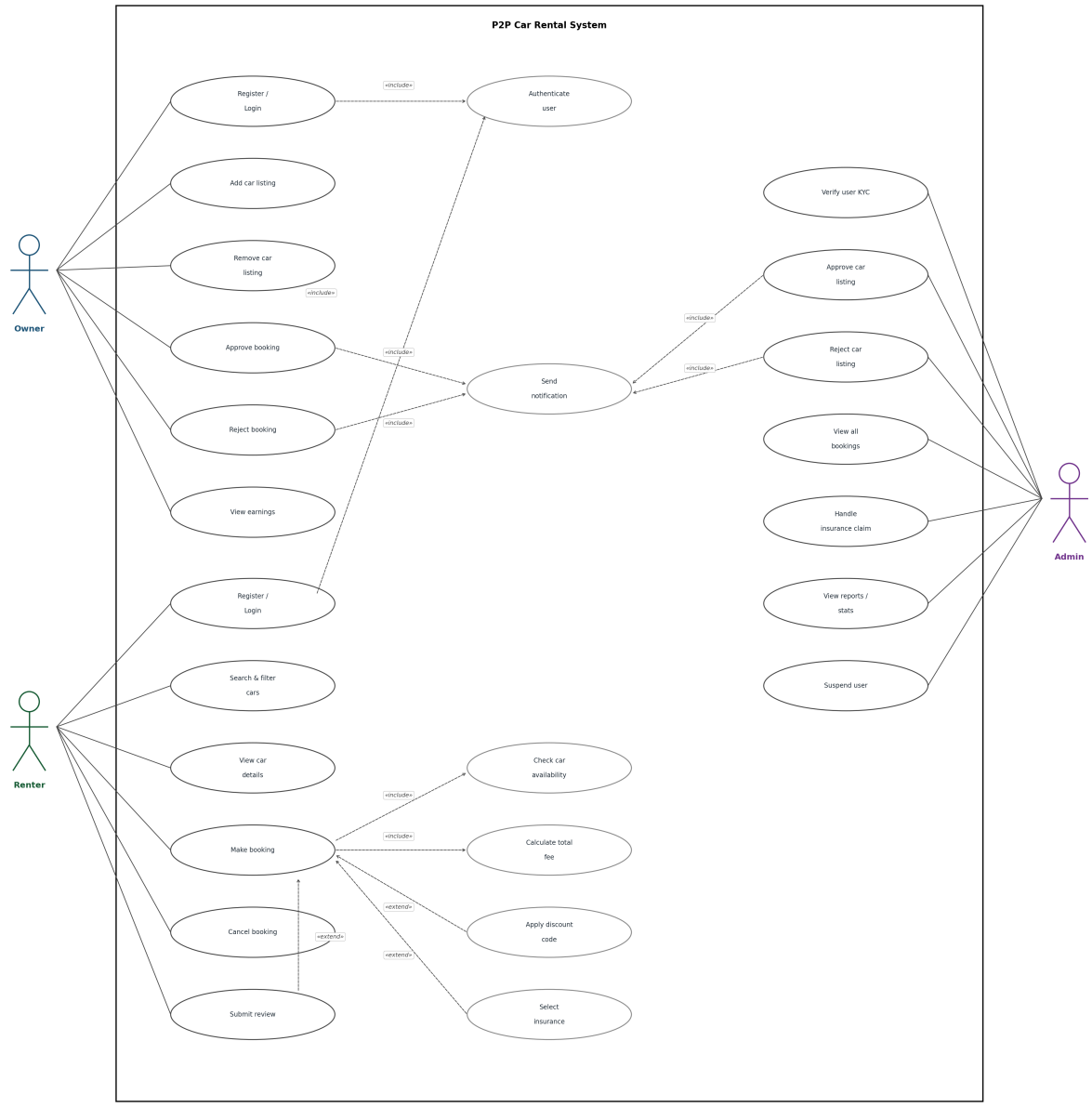

Figure 8: Use Case Diagram — 3 actors, 19 use cases.

10.3 Class Diagram

Shows the OOP design — abstract User base class with three subclasses, three pattern classes, and entity classes.

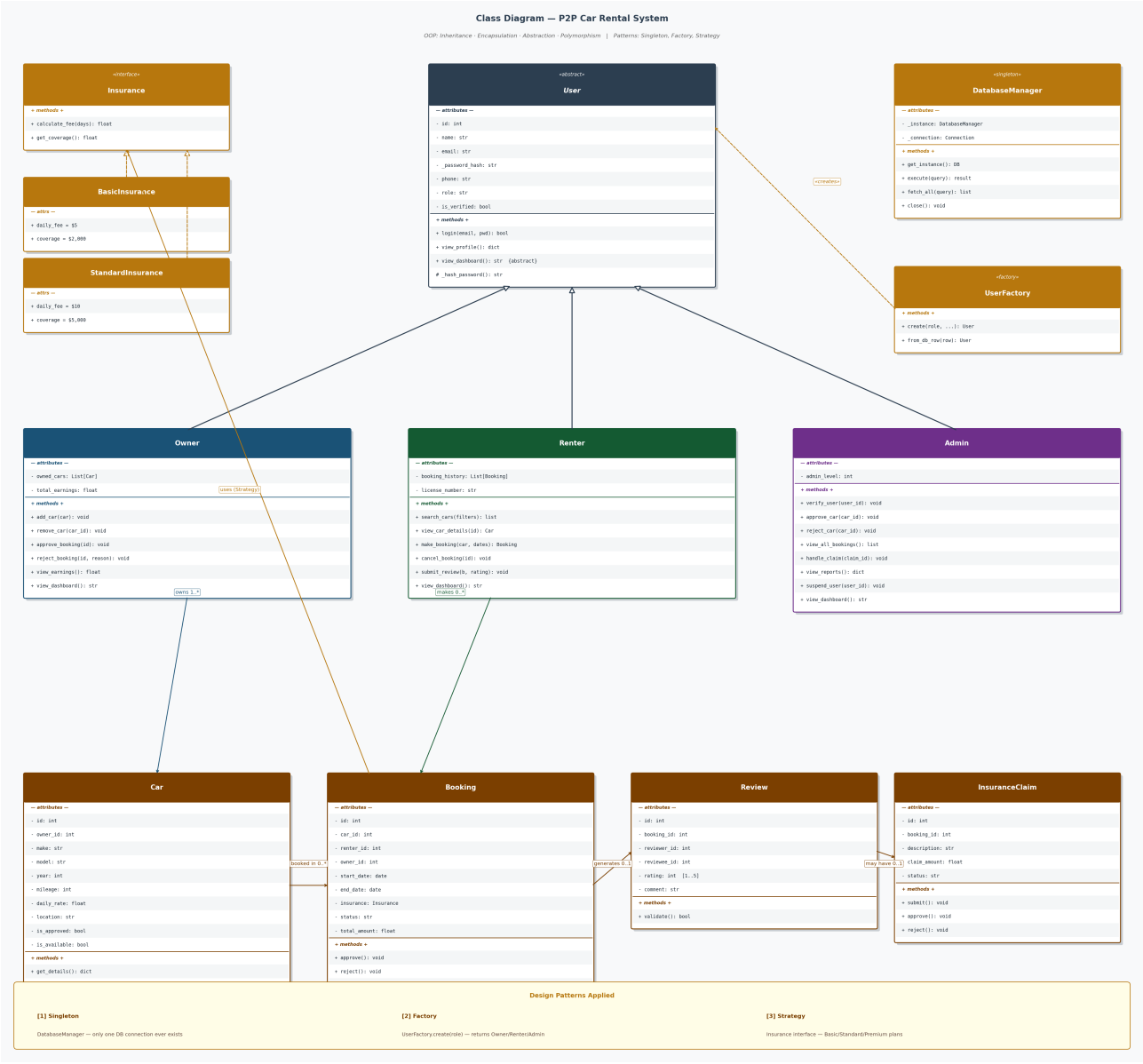

Figure 9: Class Diagram showing OOP hierarchy and patterns.

10.4 Sequence Diagram — Make Booking

Traces the most complex use case in the system, showing the order of messages between participants over time.

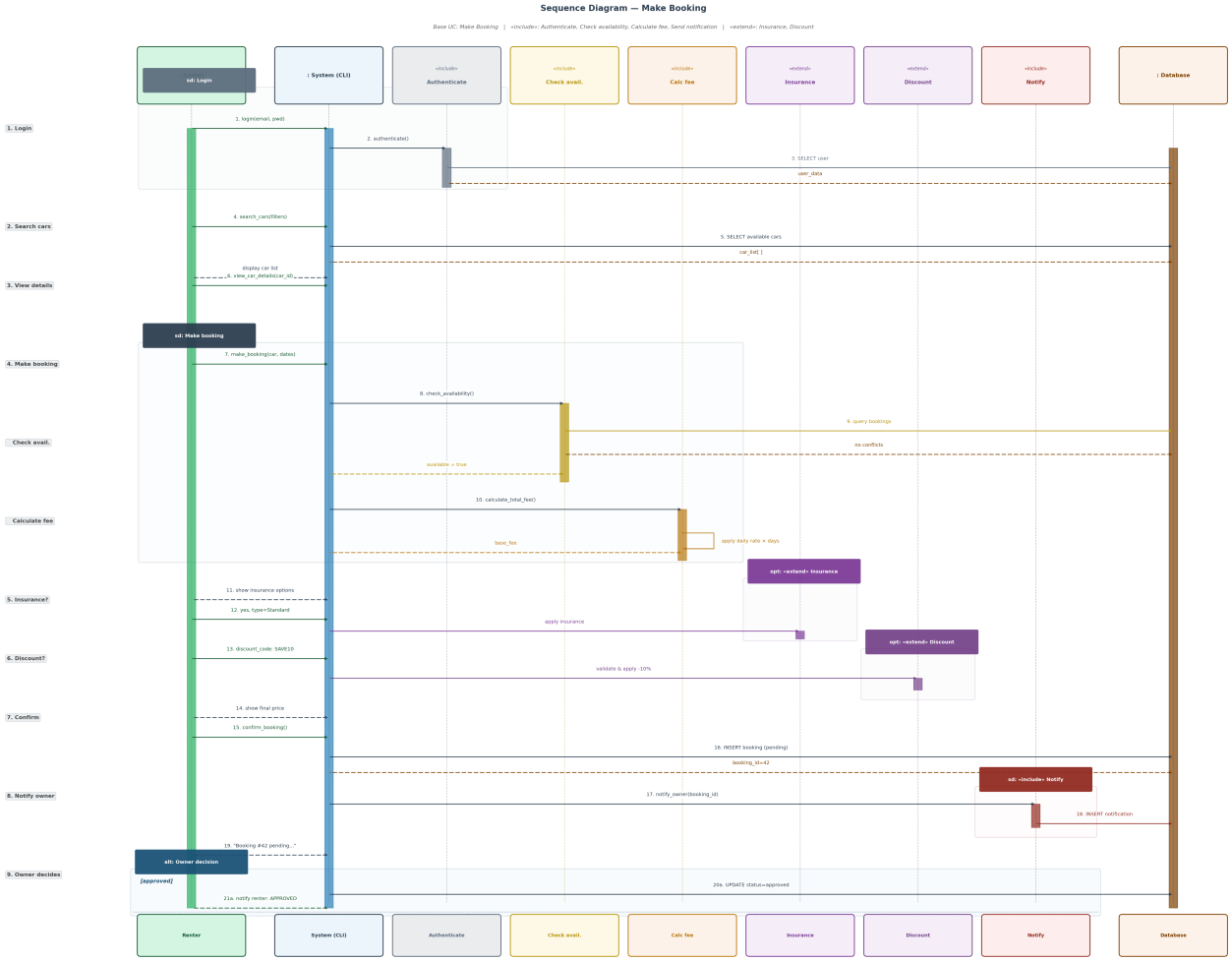

Figure 10: Sequence Diagram for the Make Booking use case.

10.5 Activity Diagram — Make Booking

The Renter's end-to-end booking flow with fork/join for parallel includes and optional extensions.

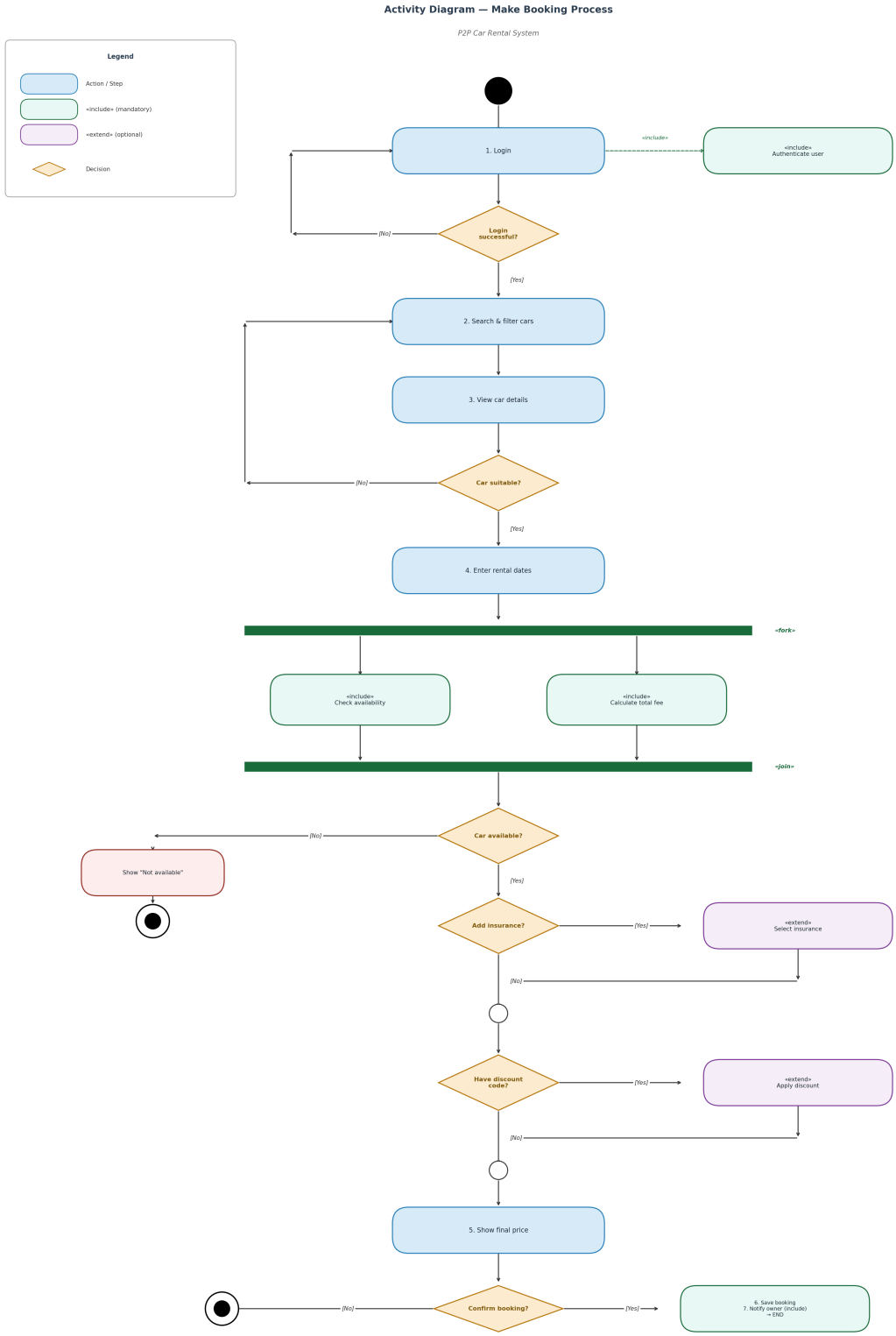

Figure 11: Activity Diagram — Make Booking process.

10.6 Activity Diagram — Add Car Listing

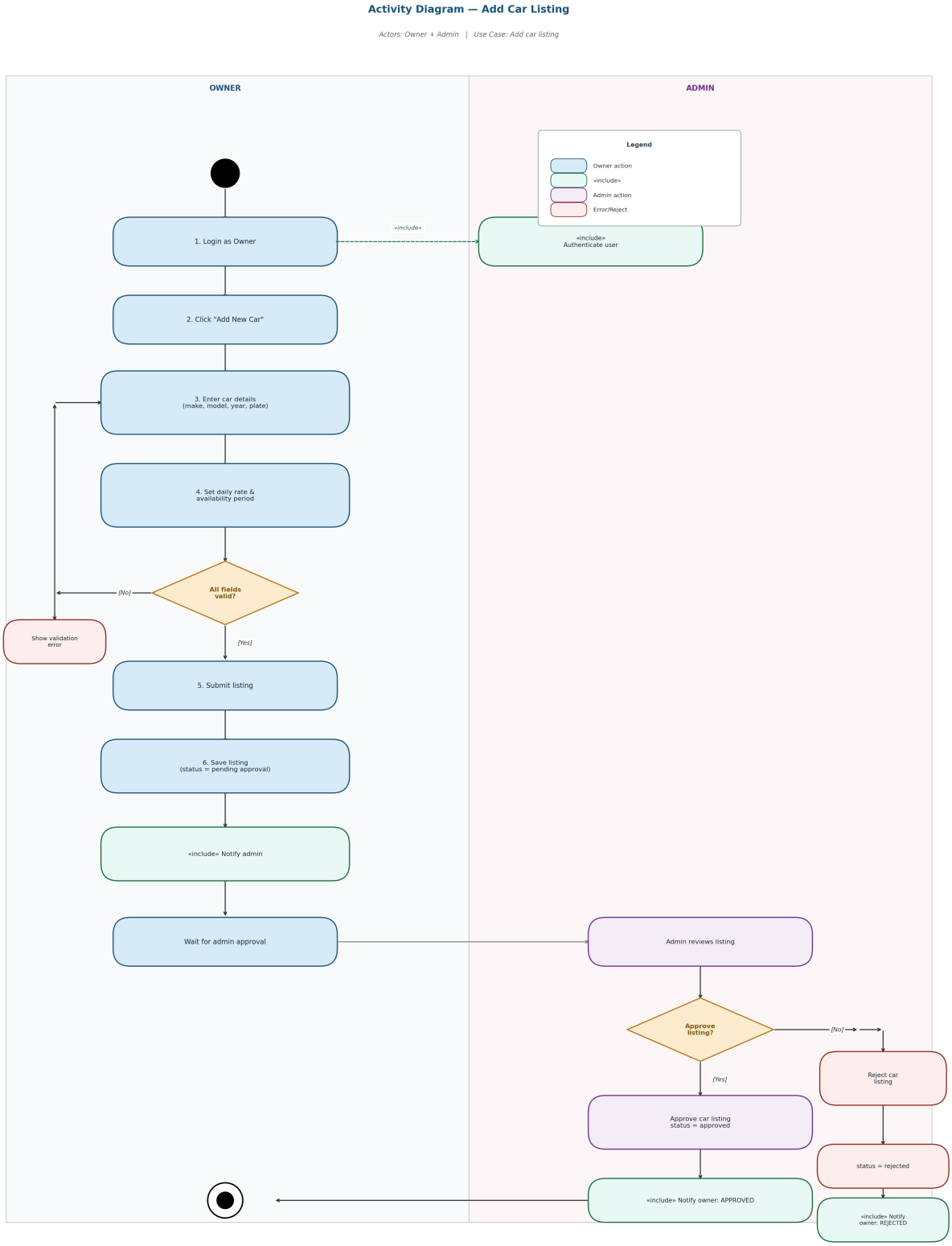

Figure 12: Activity Diagram — Add Car Listing (Owner submits, Admin approves).

10.7 Activity Diagram — Approve / Reject Booking

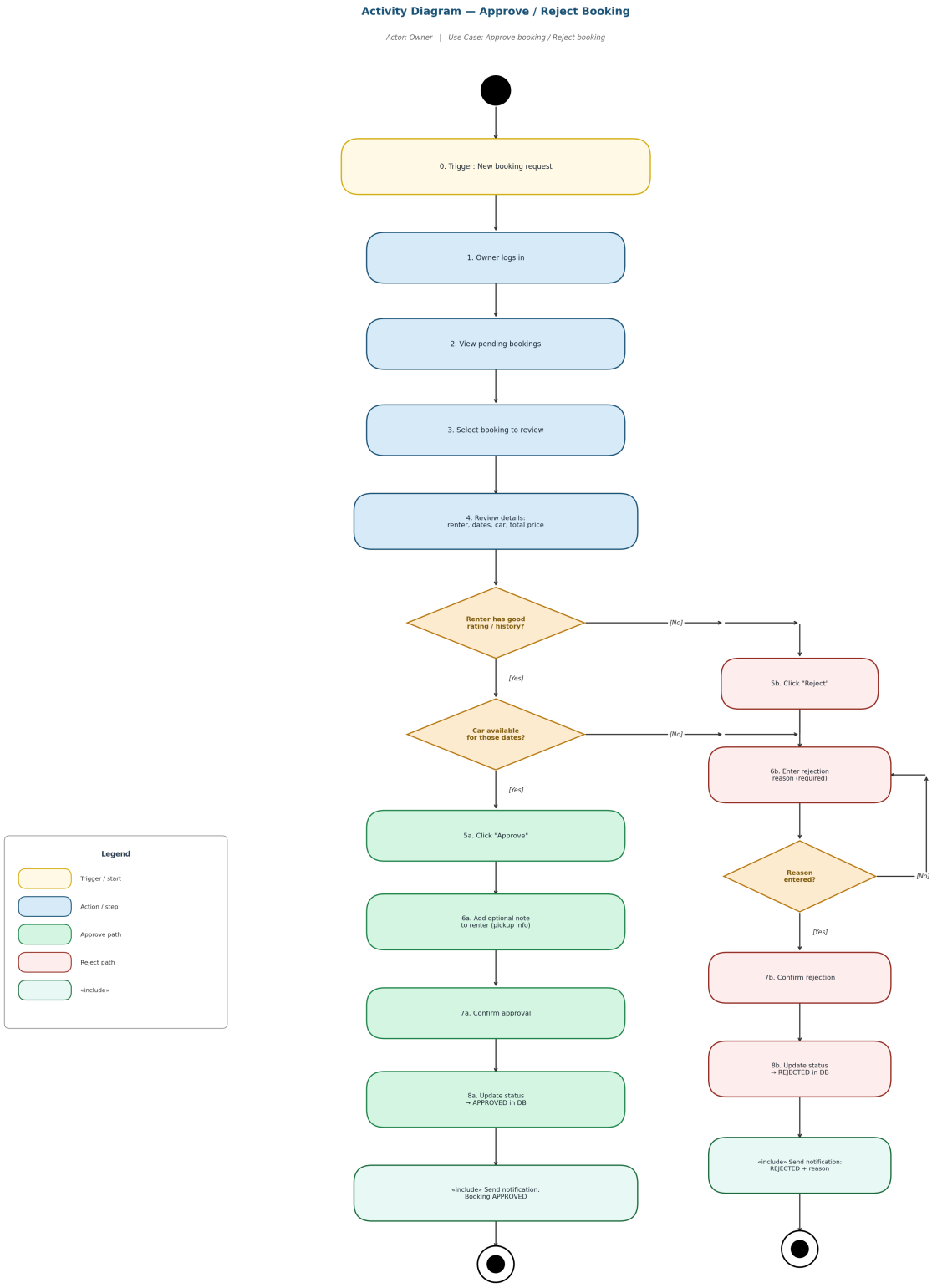

Figure 13: Activity Diagram — Owner approves or rejects a booking request.

10.8 Activity Diagram — Handle Insurance Claim

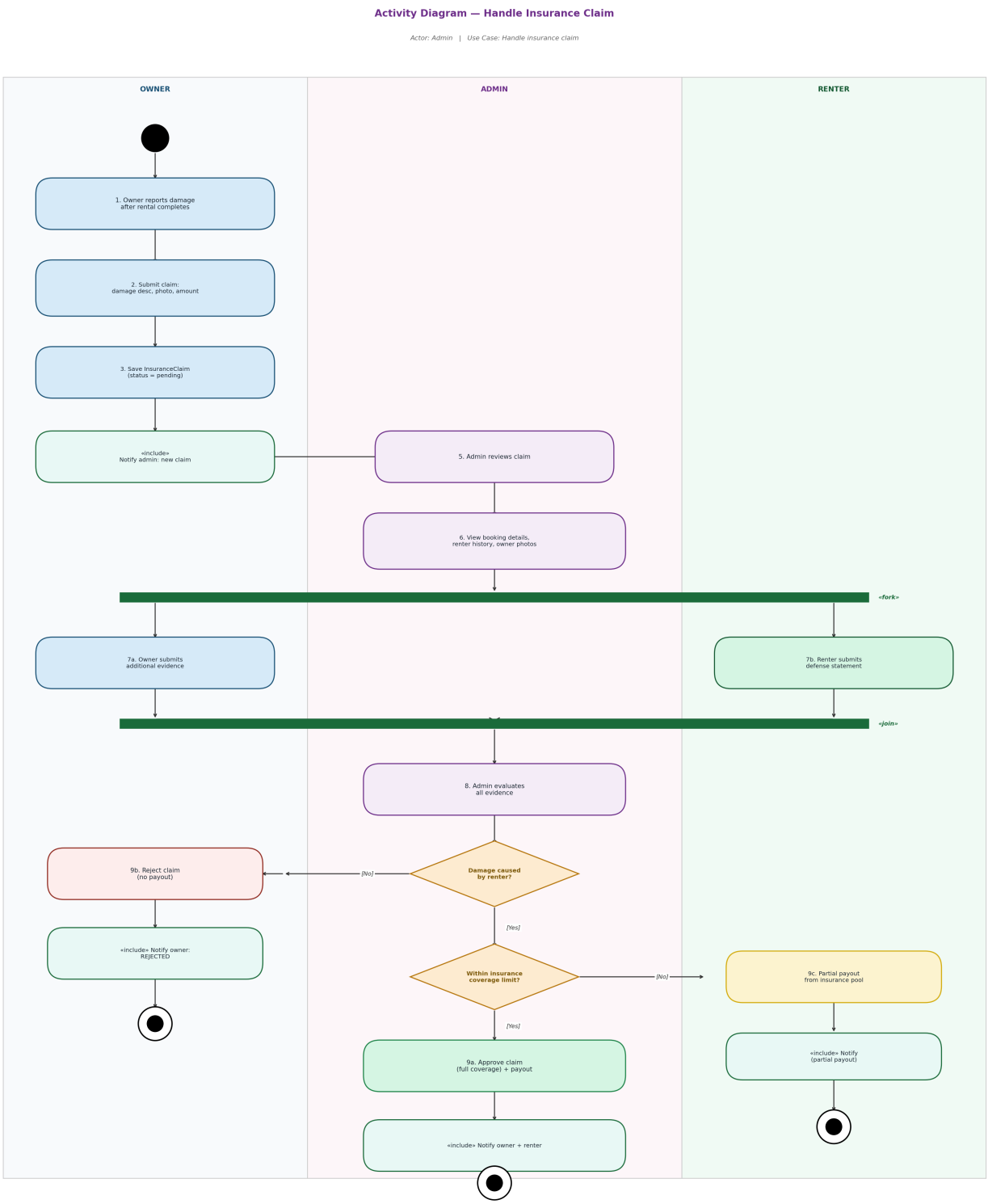

Figure 14: Activity Diagram — Insurance claim with 3 swimlanes and fork/join.

11. Testing

The system was tested through manual integration tests covering the full lifecycle. Each test verifies end-to-end behavior rather than individual units.

11.1 Test Scenarios

#	Scenario	Result
1	Default admin is auto-created on first run	PASS
2	Owner registration with duplicate email	PASS (rejected)
3	Admin verifies pending users	PASS
4	Owner adds car; admin approves it	PASS
5	Renter searches by location & price	PASS
6	Renter makes 10-day booking with insurance + discount	PASS
7	Pricing calculation (verified manually)	PASS (see below)
8	Date overlap detection (booking conflict)	PASS (rejected)
9	Owner approves booking; renter notified	PASS
10	Renter submits review (rating 1-5)	PASS
11	Renter submits invalid rating (e.g. 6)	PASS (rejected)
12	Admin views platform-wide stats	PASS
13	Login with wrong password	PASS (rejected)
14	Cancel booking before start date	PASS
15	Cancel already-completed booking	PASS (rejected)

11.2 Pricing Verification (Test #6)

A renter books a Toyota Camry at $60/day for 10 days, with Standard insurance and the SAVE10 discount code:

Calculation Step	Value
1. Base price (10 days × $60)	$600.00
2. Long-term discount (7+ days = 10% off)	-$60.00
3. Subtotal	$540.00

Calculation Step	Value
4. SAVE10 discount code (-10%)	-$54.00
5. Owner amount	$486.00
6. Platform fee (15% of owner amount)	+$72.90
7. Standard insurance ($10/day × 10 days)	+$100.00
8. TOTAL paid by renter	**$658.90**

The system's output exactly matches this manual calculation, confirming the pricing engine works correctly.

11.3 Performance Test

Platform statistics query (8 aggregated values) completes in **0.11 ms** thanks to the optimized single-query design and conditional aggregates. The original implementation used 8 separate queries.

12. Maintenance & Support Summary

A complete Maintenance and Support Plan is provided as a separate document. This section summarizes the three key strategies:

12.1 Software Maintenance

Four IEEE-standard maintenance types are addressed: **corrective** (bug fixes), **adaptive** (environment changes), **perfective** (improvements), and **preventive** (avoiding future failures). A 9-step issue handling workflow and a 4-tier severity SLA (Critical / High / Medium / Low) provide structure.

12.2 Versioning

Semantic Versioning 2.0.0 (MAJOR.MINOR.PATCH) is used. Every release is tagged in Git and documented in CHANGELOG.md following the "Keep a Changelog" format. Patches release weekly, minors every 4-6 weeks, majors every 12-18 months.

12.3 Backward Compatibility

Three pillars: **Data compatibility** (versioned migration scripts), **API compatibility** (stable function signatures), and **UX compatibility** (familiar workflows). A formal deprecation lifecycle (Announce → Coexist → Remove) gives users at least 6 months to adapt before breaking changes are removed.

For full details, see the separate document: *Maintenance_and_Support_Plan.pdf*

13. Conclusion

The P2P Car Rental System successfully demonstrates how object-oriented principles, design patterns, and disciplined architecture work together to produce maintainable, extensible software — even at small scale.

13.1 What Was Achieved

- All four OOP principles (encapsulation, inheritance, abstraction, polymorphism) are demonstrated in working code, not just theory.
- Three design patterns (Singleton, Factory, Strategy) are applied where they solve real problems.
- A 4-layer architecture cleanly separates presentation, business logic, domain entities, and data access.
- Environment-based configuration supports dev / test / prod workflows.
- Database indexes and optimized queries deliver measurable performance gains.
- Eight UML diagrams document the system's structure and behavior.
- Comprehensive Maintenance & Support Plan ensures long-term viability.

13.2 Lessons Learned

Building this project reinforced several engineering principles:

- **Design decisions compound over time.** Choosing a layered architecture early made adding new features (insurance, discount codes, reviews) effortless.
- **Design patterns are tools, not goals.** Each pattern was chosen because it solved a real problem — not because it sounded impressive.
- **Documentation is part of the deliverable.** UML diagrams, inline comments, and the README are not optional — they are how future maintainers (including future-me) will understand the code.
- **Configuration externalization pays off.** The ability to switch between dev/test/prod with a single environment variable made testing far easier.

13.3 Future Work

The Maintenance & Support Plan outlines a 24-month roadmap. Highlights include:

- Email notifications via SMTP (v1.1.0, Q3 2026)
- Loyalty points & tier discounts (v1.2.0, Q4 2026)
- Multi-language support (v1.3.0, Q1 2027)
- PostgreSQL backend & REST API (v2.0.0, Q2 2027)
- Mobile app via React Native (v2.1.0, Q3 2027)
- Stripe payment integration (v2.2.0, Q4 2027)
- AI-based smart matching & predictive pricing (v3.0.0, Q2 2028)

Final thought: Software is never "finished" — but it can be built well, documented thoroughly, and prepared for evolution. That has been the goal of this project.

Appendix: Default Credentials

On first run, the system seeds a default admin account so that users can verify the very first registrations.

Field	Value
Email	admin@p2p.com
Password	admin123
Role	Admin (pre-verified)

Security note: Change the default admin password immediately if deploying to production. The credentials above are for development and demonstration only.

Document Reference

Document	Purpose
Final_Report.pdf (this document)	Comprehensive overview of the project
Design_and_Architecture.pdf	UML diagrams in standalone PDF
Maintenance_and_Support_Plan.pdf	Detailed maintenance strategy
README.md	Quick-start instructions
p2p_car_rental.zip	Complete project source code

— End of Report —
